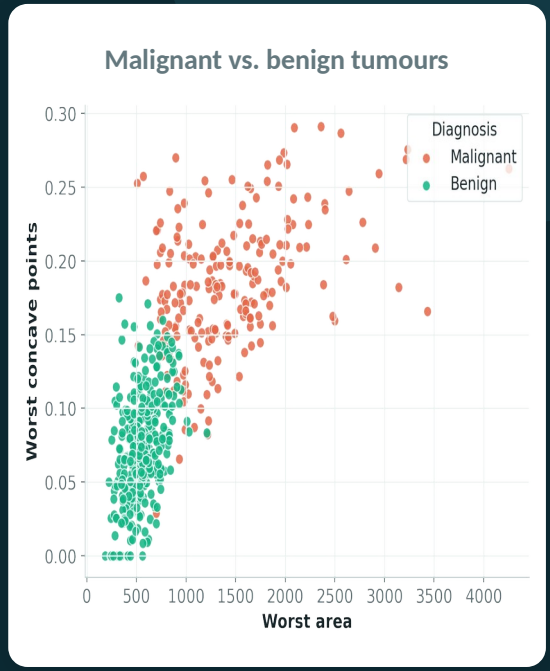
MACHINE LEARNING · ENSEMBLE METHOD

Random Forest Classifier

Diagnosing **breast tumours** by letting **100 decision trees vote.**



BUILT WITH

THEORY · 1

What is a Random Forest?

A Random Forest is an **ensemble** — it grows many decision trees and lets them **vote.** One tree can be wrong; a crowd of varied trees is usually right.

Wisdom of the crowd

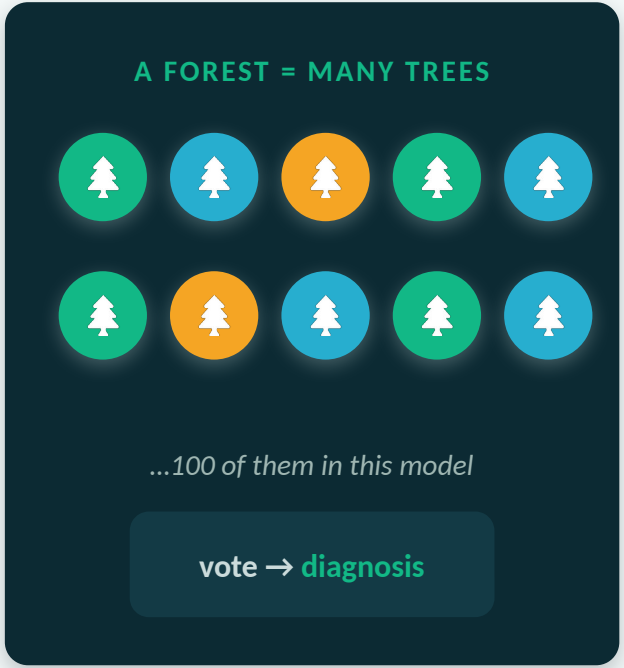
Many trees together beat any single tree — errors tend to cancel out.

Built on randomness

Each tree sees a random slice of rows and features, so they think differently.

Decides by majority

Every tree casts a vote; the most-voted class is the forest's answer.



THEORY · 2

Bagging: each tree sees a random sample

1

Draw a random sample

Pick rows at random (with repeats) — a fresh ‘bootstrap’ for each tree.

2

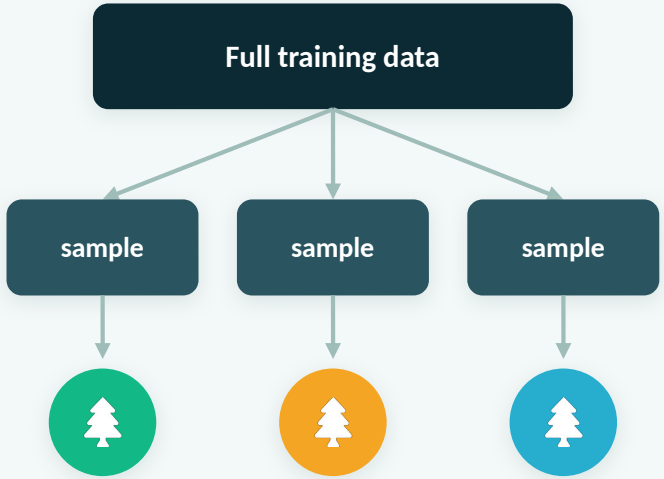
Pick random features

At each split a tree may only consider a random subset of the 30 features.

3

Grow many trees

Repeat 100 times, so every tree is a little different from the others.



random rows + random features each time

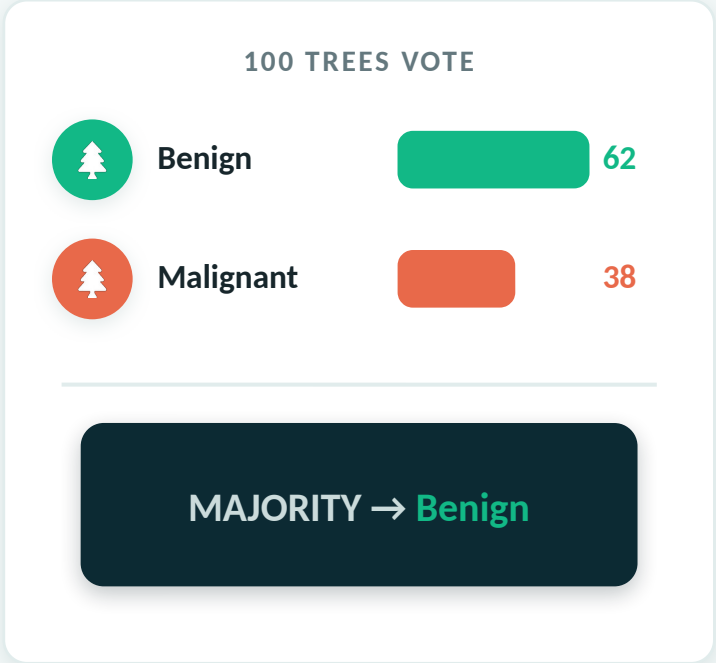
THEORY · 3

Every tree votes — the majority wins

To classify a new tumour, the forest runs it through **all 100 trees** and tallies their votes. The class with the most votes becomes the final prediction.

More robust. A wrong vote from one odd tree is outweighed by the rest.

Less overfitting. Averaging many varied trees smooths out their individual quirks.



THEORY · 4

Tuning the forest

n_estimators = 100 (more trees)

More trees give a steadier, more reliable vote. Returns flatten out after a point, but 100 is a solid, common default.

max_depth = 5 (shallow trees)

Capping depth keeps each tree simple so it can't memorise noise — the ensemble handles the complexity instead.

THIS MODEL

100

decision trees

max depth = 5

random_state = 42

THEORY · 5

The Breast-Cancer Dataset

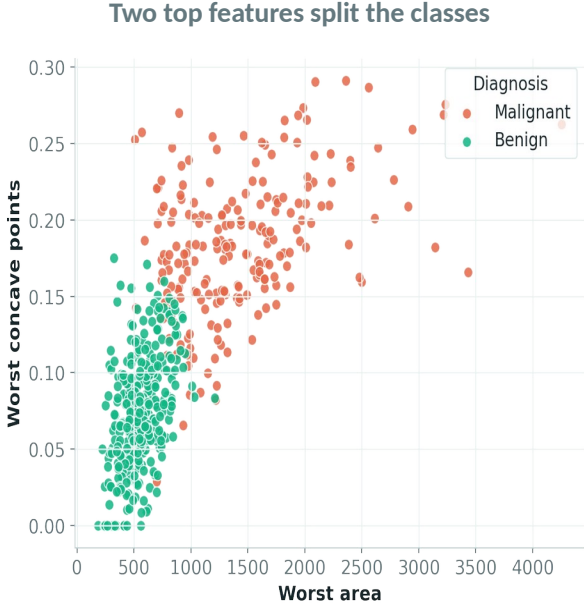
569	30	2
tumours	features	classes

EXAMPLE FEATURES (30 IN TOTAL → X)

- mean radius
- mean texture
- worst area
- worst concave points

THE DIAGNOSIS (TARGET → Y)

- 0 · Malignant (212)
- 1 · Benign (357)



THE WORKFLOW

7 steps from data to a diagnosis

STEP 1

Import

STEP 2

Load data

STEP 3

Split

STEP 4

Build forest

STEP 5

Predict

STEP 6

Accuracy

STEP 7

Importances

Next: every step explained line by line →

STEP 1 · CODE

Import the tools

imports.py

```python
# the libraries we need
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import (
    train_test_split)
from sklearn.ensemble import (
    RandomForestClassifier)
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report)
```

LINE BY LINE

- **pandas / numpy**
  handle data as tables & arrays.
- **matplotlib / seaborn**
  draw the importance chart.
- **load_breast_cancer**
  the sample medical dataset.
- **train_test_split**
  split into train & test.
- **RandomForest + metrics**
  the model & scoring tools.

STEP 2 · CODE

Load the dataset

load.py

```
# load the breast-cancer dataset
data = load_breast_cancer()

# X = feature table, Y = 0/1 labels
X = pd.DataFrame(data.data,
    columns=data.feature_names)
Y = data.target
```

LINE BY LINE

- **`load_breast_cancer()`**
  569 tumours × 30 measurements.
- **`pd.DataFrame(...)`**
  puts features in a labelled table.
- **`Y = data.target`**
  0 = malignant, 1 = benign.

STEP 3 · CODE

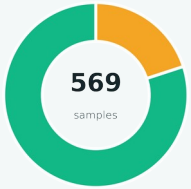
Split: train vs. test

split.py

```
# 80% train, 20% test
X_train, X_test, Y_train, Y_test = (
    train_test_split(
        X, Y,
        test_size=0.2,
        random_state=42))
```

LINE BY LINE

- **`test_size=0.2`**
  hold back 20% to test the forest.
- **`random_state=42`**
  fixes the split so runs match.
- **`4 outputs`**
  train + test inputs and labels.



80% train
20% test

STEP 4 · CODE

Build & train the forest

train.py

```
# a forest of 100 trees, depth 5
rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42)

# fit = grow all 100 trees
rf_model.fit(X_train, Y_train)
```

LINE BY LINE

- **`n_estimators=100`**
  builds 100 different decision trees.
- **`max_depth=5`**
  keeps each tree shallow — limits overfitting.
- **`.fit(...)`**
  grows every tree on the training data.

Each tree trains on a random sample

STEP 5 · CODE

Make predictions

predict.py

```
# every tree votes; majority wins
Y_pred = rf_model.predict(X_test)
```

LINE BY LINE

- **`.predict(X_test)`**
  runs every tree and takes the majority vote.

- **`Y_pred`**
  the predicted label (malignant / benign) per tumour.

STEP 6 · CODE + OUTPUT

Score the model

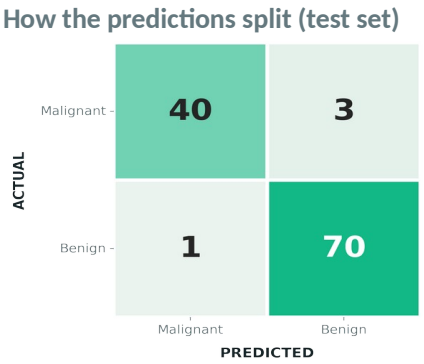
evaluate.py

```
# accuracy as a percentage
accuracy = accuracy_score(
    Y_test, Y_pred) * 100
print(accuracy)
```

- `accuracy_score` fraction predicted correctly.
- `* 100` shown as a percentage.

96.49% accuracy on 114 unseen tumours

(100 trees · max_depth = 5)



3 + 1 mistakes out of 114

STEP 7 · CODE + OUTPUT

Which features matter (plt.show)

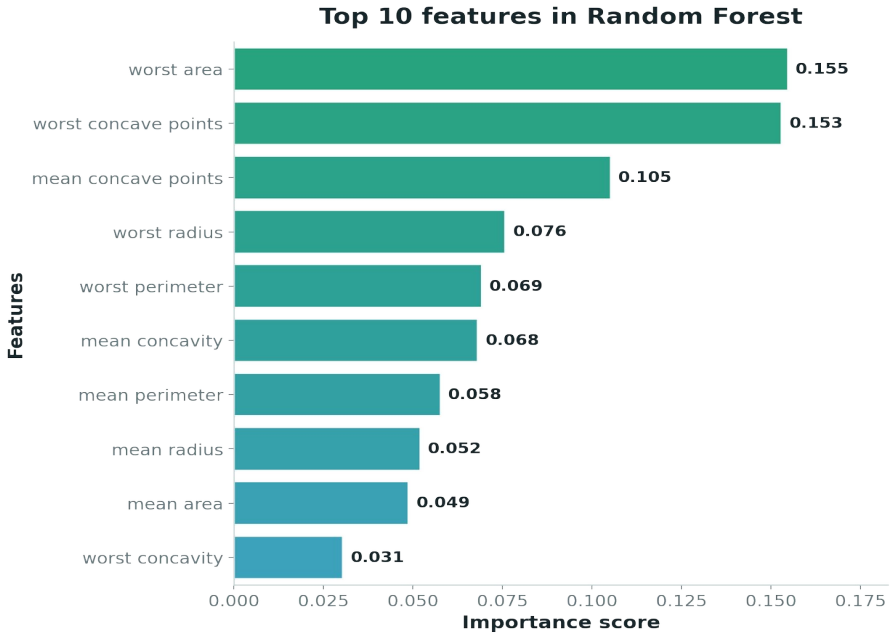
importance.py

```python
# importance score per feature
importances = (
    rf_model.feature_importances_)

# sort the scores, high to low
feature_imp = pd.Series(importances,
    index=data.feature_names
    ).sort_values(ascending=False)

# plot the top 10
plt.figure(figsize=(18, 6))
sns.barplot(x=feature_imp[:10],
    y=feature_imp.index[:10])
plt.title("Top 10 features ...")
plt.show()
```

THE OUTPUT → top-10 feature importances



RECAP

What we built

Wisdom of many trees

100 shallow trees each vote; the majority gives a stabler answer than one tree.

Randomness adds diversity

Random rows and features per tree mean their individual errors cancel out.

Ranks the features

The forest flags ‘worst area’ & ‘worst concave points’ as the key signals.

96%

test accuracy

Random Forest · 100 trees